

Here we have two classes, `Student` and `Teacher`. The main code instantiates one `Teacher` object ❹, and its `__init__()` method creates three instances of the `Student` class ❷, one each for Joe, Sue, and Chris. Therefore, after starting up, the `Teacher` object has three instance variables that are `Student` objects. The output from the first part is:

---

```
Creating the Teacher object
Creating Student object Joe
Creating Student object Sue
Creating Student object Chris
```

---

Next, the main code uses the `del` statement to delete the `Teacher` object ❺. Since we wrote a `__del__()` method in the `Teacher` class ❸, that method of the `Teacher` *object* is called—which, for demonstration purposes, only prints a message.

When the `Teacher` object is deleted, Python sees that it contains three other objects (the three `Student` objects). So, Python lowers the reference count of each of those objects from 1 to 0.

Once this happens, the `__del__()` method of the `Student` objects is called ❶, and each outputs a message. The memory used by all three of the `Student` objects is then marked as garbage. The output from the end of the program is:

---

```
In the __del__ method for Teacher
In the __del__ method for student: Joe
In the __del__ method for student: Sue
In the __del__ method for student: Chris
```

---

Because Python keeps track of reference counts for all objects, you rarely, if ever, have to worry about memory management in Python and rarely need to include a `__del__()` method. However, you might consider using a `del` statement to explicitly tell Python to delete objects that use a very large amount of memory when you are no longer using them. For example, you might want to delete an object that loads a large number of records from a database, or loads many images, when you are done using it. Also, it is not guaranteed that Python will call the `__del__()` method when a